Setup

Initial environment setup and verification of the supplied image assets, including their dimensions, colour modes, and file sizes.

```
# Uncomment if needed, then run once:
# %pip install pillow stepic

from pathlib import Path
from PIL import Image
import hashlib

ASSET_DIR = Path("Lab7")
LOGO = ASSET_DIR / "compay_logo.png"
PROFILE = ASSET_DIR / "profile.png"
PYTHON_IMG = ASSET_DIR / "python.png"
OUTPUT_DIR = Path("Lab8_outputs")
OUTPUT_DIR.mkdir(exist_ok=True)

for path in [LOGO, PROFILE, PYTHON_IMG]:
    if not path.exists():
        raise FileNotFoundError(f"Missing {path}. Check where you extracted Lab8.zip")
    with Image.open(path) as image:
        print(path.name, image.size, image.mode, path.stat().st_size, "bytes")
```

✓ 0.0s

```
compay_logo.png (1024, 1024) RGBA 1029126 bytes
profile.png (1408, 768) RGBA 2424849 bytes
python.png (100, 100) RGBA 9427 bytes
```

Check in 0:
RGBA stands for Red, Green, Blue, and Alpha. Red, Green, and Blue represent the colour channels of an image, while Alpha represents the transparency or opacity channel.

A forensic analyst might calculate a file's SHA-256 hash to create a digital fingerprint of the file. The hash can be used to check whether a file has changed by comparing its hash before and after an operation. If the hashes differ, the file's bytes are different. However, a different hash does not by itself prove that a file contains hidden information.

Original SHA-256 hashes recorded for the company logo and profile images to establish baseline file integrity before steganographic modifications.

```
def sha256_file(path):
    """Calculate the SHA-256 digest of a file's exact bytes."""
    with open(path, "rb") as file:
        return hashlib.sha256(file.read()).hexdigest()

# Save baseline hashes now; we will use these values again in Part 5.
original_logo_hash = sha256_file(LOGO)
original_profile_hash = sha256_file(PROFILE)

print("Original logo SHA-256:    ", original_logo_hash)
print("Original profile SHA-256:", original_profile_hash)
```

✓ 0.0s

```
Original logo SHA-256:    64303ae8e227a463b9d1c9c27359d228bf0874c1284535f6f80f3e0802a6a6e0
Original profile SHA-256: 292d9a5bcf8b10017abc1e633a86ae0ddbac4e5c2156e84d6dbf5307b3df5850
```

Part 2

LSB manipulation using the decimal value 201, demonstrating how changing the least significant binary bit changes the decimal value.

Worked example: set and read one bit

```
def set_LSB(value, bit):
    """Return an integer with its least significant bit set to 0 or 1."""
    if bit not in ("0", "1"):
        raise ValueError("bit must be '0' or '1'")
    return (value & 254) if bit == "0" else (value | 1)

def get_LSB(value):
    return str(value & 1)

value = 150
for bit in ("0", "1"):
    changed = set_LSB(value, bit)
    print(value, format(value, "08b"), "→", changed, format(changed, "08b"), "LSB", get_LSB(changed))

assert set_LSB(150, "1") == 151
assert get_LSB(151) == "1"
```

✓ 0.0s

```
150 10010110 → 150 10010110 LSB 0
150 10010110 → 151 10010111 LSB 1
```

```
# TODO: Run two examples with value = 201 and print binary before/after.
value = 201
# your code here

print("Original:", value, format(value, "08b"))

for bit in ("0", "1"):
    changed = set_LSB(value, bit)

    print(
        "LSB:", bit,
        "| Decimal:", changed,
        "| Binary:", format(changed, "08b")
    )
```

✓ 0.0s

```
Original: 201 11001001
LSB: 0 | Decimal: 200 | Binary: 11001000
LSB: 1 | Decimal: 201 | Binary: 11001001
```

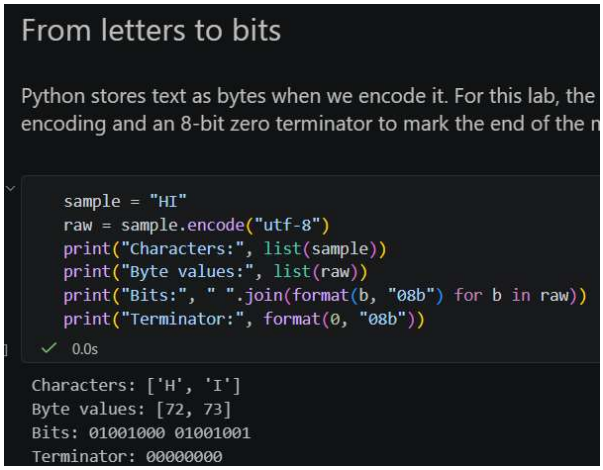
Explain why set_LSB(201, "1") does not change the value

The decimal value 201 is 11001001 in binary. Its least significant bit (the final bit) is already 1. Therefore, setting the LSB to "1" does not change any bits, so the value remains 201.

What is the maximum numerical difference between an original channel and its LSB-modified value?

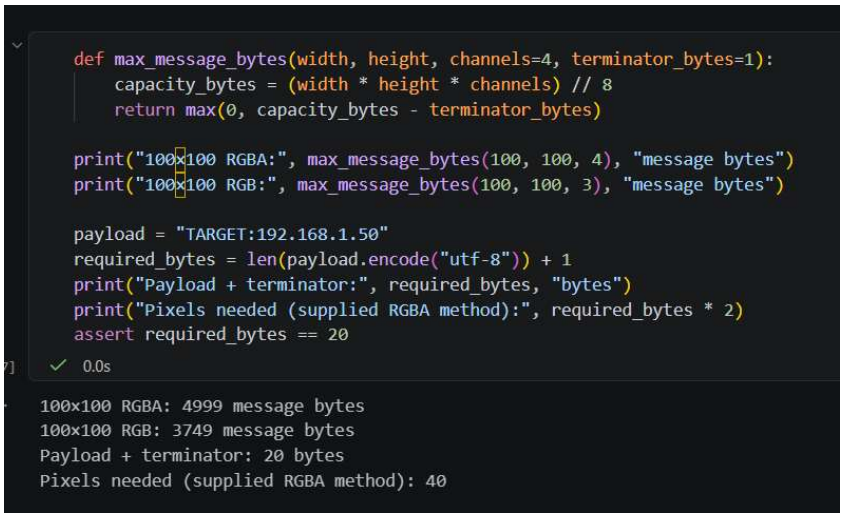
The maximum numerical difference is 1, because changing the least significant bit changes the channel value by only 1. For example, changing 201 (11001001) to 200 (11001000) gives a difference of 1.

Converts letters to bits



Part 2

Calculation of the maximum message capacity for RGB and RGBA images and the pixel requirements for the fictional target message.



How many message bytes fit in a 100 × 100 RGBA image using this exact method? How would an RGB-only, one-LSB-per-channel method differ?

A 100 × 100 RGBA image has 10,000 pixels. Each pixel contains four channels, and each channel can store one bit using the LSB method.

- 10,000 × 4 = 40,000 available bits.
- 40,000 ÷ 8 = 5,000 bytes.
- Subtract one byte for the null terminator.
- Maximum message capacity = 4,999 bytes.

With an RGB-only method, each pixel has three channels instead of four. This provides 30,000 bits, or 3,750 bytes. After reserving one byte for the terminator, the capacity is 3,749 bytes.The

RGB-only method does not modify the alpha channel. It also provides fewer bits per pixel, so its capacity is lower.

Part 3

Testing the supplied LSB functions by embedding the message “HI” into the Python image and successfully recovering the original message.



Why does the reference code call .convert("RGBA")?
The reference code converts the image to RGBA so that every pixel has four consistent channels: red, green, blue, and alpha. This allows the encoder to access four values per pixel and modify the least significant bit of each channel.

Why are two pixels used per byte?
Each RGBA pixel has four channels, and the encoder stores one bit in each channel. One pixel therefore stores four bits. Two pixels provide eight bits, which make up one byte or one character.

What does message += chr(0) accomplish?
It adds a null character to the end of the message. This acts as an end marker so that the decoder knows when to stop extracting the hidden message.

What happens if the payload exceeds image capacity? How could your code detect this before editing any pixels?
If the payload exceeds the image capacity, the encoder may try to access pixel positions that do not exist, causing an error. The code can prevent this by calculating the number of pixels required for the message and comparing it with the total number of pixels available before embedding anything.

Why is PNG preferable to JPEG for preserving exact LSB values?
PNG uses lossless compression, which preserves the image's pixel values. JPEG uses lossy compression, which can change pixel values and therefore alter the least significant bits containing the hidden message.

Implementation of a validated LSB encoder, including input validation, image capacity checking, and creation of the company logo steganographic image.

```
        hide_message(
            str(source_path),
            message,
            str(output_path)
        )

        return output_path


    LOGO_STEGO = encode_checked(
        LOGO,
        "TARGET:192.168.1.50",
        OUTPUT_DIR / "company_logo_stego.png"
    )

    print("Output created:", LOGO_STEGO)
```

[9] ✓ 0.3s

```
Available pixels: 1048576
Required pixels: 40
Output created: Lab8_outputs\company_logo_stego.png
```

Decoder. Verification of the hidden fictional target label using both the manually implemented LSB decoder and the supplied reference decoder.

```
            second_bits = "".join(
                str(channel & 1) for channel in second
            )
            bits = first_bits + second_bits
            value = int(bits, 2)

            if value == 0:
                break
            decoded_bytes.append(value)

    return decoded_bytes.decode("utf-8")

manual_message = decode_manual(LOGO_STEGO)

reference_message = extract_message(str(LOGO_STEGO))

print("Manual decoder:", manual_message)
print("Reference decoder:", reference_message)

assert manual_message == "TARGET:192.168.1.50"
assert reference_message == "TARGET:192.168.1.50"

print("Both decoders successfully recovered the message.")
```

1] ✓ 0.2s Python

```
Manual decoder: TARGET:192.168.1.50
Reference decoder: TARGET:192.168.1.50
Both decoders successfully recovered the message.
C:\Users\rkory\AppData\Local\Temp\ipykernel_9824\4210582223.py:3: DeprecationWarning: Image.Image.getdata is deprecated and will be removed in Pillow 14 (2027-10-15). Use get_flattene
  pixels = list(image.convert("RGBA").getdata())
```

Robustness Test

Two robustness tests confirmed that the validated encoder rejects bad input before editing any pixels. An empty message was rejected with "Message must not be empty", and a 6,000-character message was rejected for the 100 x 100 python.png image, which has 10,000 pixels available but needs 12,002, with "Message exceeds image capacity". Neither failed attempt created an output file

```python
# Robustness tests

# Test 1: an empty message must be rejected.
try:
    encode_checked(LOGO, "", OUTPUT_DIR / "empty_test.png")
    print("Test 1 FAILED: empty message was accepted")
except ValueError as error:
    print("Test 1 PASSED: empty message rejected:", error)
assert not (OUTPUT_DIR / "empty_test.png").exists(), "Failed encode created a file"

# Test 2: a message too large for the 100 x 100 image must be rejected.
try:
    encode_checked(PYTHON_IMG, "A" * 6000, OUTPUT_DIR / "oversize_test.png")
    print("Test 2 FAILED: oversize message was accepted")
except ValueError as error:
    print("Test 2 PASSED: oversize message rejected:", error)
assert not (OUTPUT_DIR / "oversize_test.png").exists(), "Failed encode created a file"
```

✓ 0.0s

```
Test 1 PASSED: empty message rejected: Message must not be empty
Available pixels: 10000
Required pixels: 12002
Test 2 PASSED: oversize message rejected: Message exceeds image capacity
```

Part 4

Byte Payload

```python
ports = [80, 443, 3478, 5060]
port_text = ",".join(str(port) for port in ports)
port_bytes = port_text.encode("utf-8")
print("Message:", port_text)
print("Bytes:", port_bytes)
assert port_text == "80,443,3478,5060"
```

✓ 0.0s

```
Message: 80,443,3478,5060
Bytes: b'80,443,3478,5060'
```

Encoding and decoding the fictional port list using Stepic, confirming that the recovered message matches the original payload.

```python
# If Stepic is installed:
# import stepic
# with Image.open(PROFILE) as source:
#     rgb_image = source.convert("RGB")
#     # TODO: call stepic.encode(rgb_image, port_bytes)
#     # TODO: save returned image to OUTPUT_DIR / "profile_secret.png"
#     pass
import stepic

with Image.open(PROFILE) as source:

    rgb_image = source.convert("RGB")

    encoded_image = stepic.encode(
        rgb_image,
        port_bytes
    )

    profile_stego = OUTPUT_DIR / "profile_secret.png"

    encoded_image.save(
        profile_stego,
        "PNG"
    )

print("Stepic output created:", profile_stego)
```

[13] ✓ 0.1s

```
Stepic output created: Lab8_outputs\profile_secret.png
```

```python
# TODO: Open profile_secret.png, call stepic.decode, normalize to text if needed.
# TODO: assert recovered_port_text == port_text
# Print the recovered text.
with Image.open(profile_stego) as image:
    recovered_ports = stepic.decode(image)

if isinstance(recovered_ports, bytes):
    recovered_port_text = recovered_ports.decode("utf-8")
else:
    recovered_port_text = recovered_ports

print("Recovered port list:", recovered_port_text)

assert recovered_port_text == port_text

print("Stepic message successfully recovered.")
```

] ✓ 0.0s

```
Recovered port list: 80,443,3478,5060
Stepic message successfully recovered.
```

Analysis: Why is it important to verify extraction rather than assuming a successful save means the payload survived?

Saving an image successfully only confirms that an output file was created. It does not prove that the hidden message was embedded correctly or can be recovered. Verifying extraction confirms that the recovered message matches the original payload. This helps identify encoding errors, compatibility problems, or data loss.

Part 5

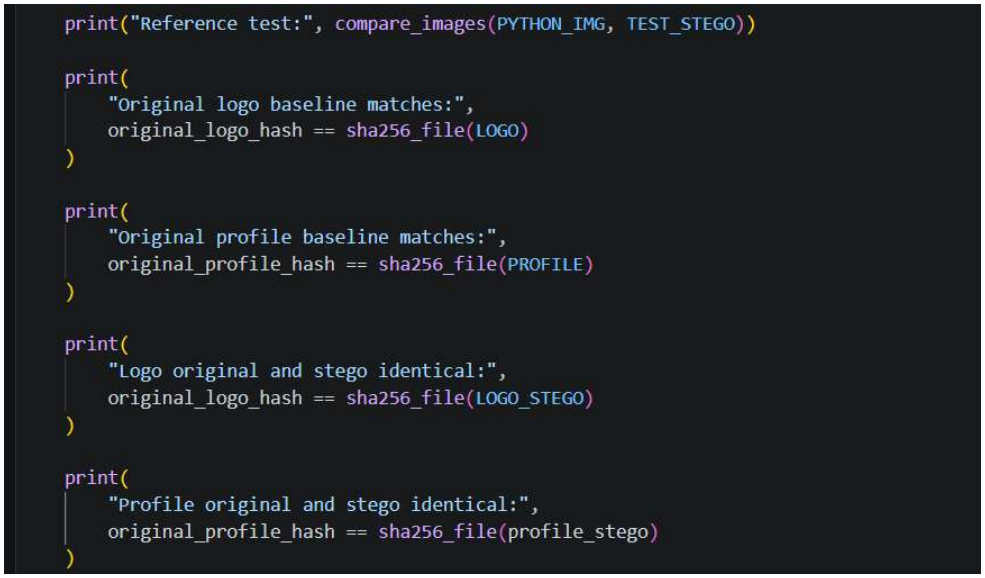
Worked comparison function, and comparing the SHA256 hashes



```
def compare_images(original_path, stego_path):
    with Image.open(original_path) as original, Image.open(stego_path) as stego:
        a = list(original.convert("RGBA").getdata())
        b = list(stego.convert("RGBA").getdata())
        if original.size != stego.size:
            raise ValueError("Image dimensions differ")
    changed_pixels = sum(p != q for p, q in zip(a, b))
    changed_channels = sum(x != y for p, q in zip(a, b) for x, y in zip(p, q))
    return {
        "original_bytes": Path(original_path).stat().st_size,
        "stego_bytes": Path(stego_path).stat().st_size,
        "original_sha256": sha256_file(original_path),
        "stego_sha256": sha256_file(stego_path),
        "same_file_bytes": sha256_file(original_path) == sha256_file(stego_path),
        "changed_pixels": changed_pixels,
        "changed_channels": changed_channels,
        "total_pixels": len(a),
    }

print("Reference test:", compare_images(PYTHON_IMG, TEST_STEGO))
```

```
✓ 0.0s                                                                    Python
Reference test: {'original_bytes': 9427, 'stego_bytes': 9471, 'original_sha256': 'b1f47b31ac5fb11d45b17576d57775931191da73bec566b02074a119bb6700fd', 'stego_sha256': '10aa54fdbce9fa694
C:\Users\rkory\AppData\Local\Temp\ipykernel_9824\40363051.py:3: DeprecationWarning: Image.Image.getdata is deprecated and will be removed in Pillow 14 (2027-10-15). Use get_flattened
  a = list(original.convert("RGBA").getdata())
C:\Users\rkory\AppData\Local\Temp\ipykernel_9824\40363051.py:4: DeprecationWarning: Image.Image.getdata is deprecated and will be removed in Pillow 14 (2027-10-15). Use get_flattened
  b = list(stego.convert("RGBA").getdata())
```



```
print("Reference test:", compare_images(PYTHON_IMG, TEST_STEGO))

print(
    "Original logo baseline matches:",
    original_logo_hash == sha256_file(LOGO)
)

print(
    "Original profile baseline matches:",
    original_profile_hash == sha256_file(PROFILE)
)

print(
    "Logo original and stego identical:",
    original_logo_hash == sha256_file(LOGO_STEGO)
)

print(
    "Profile original and stego identical:",
    original_profile_hash == sha256_file(profile_stego)
)
```



```
Reference test: {'original_bytes': 9427, 'stego_bytes': 9471, 'original_sha256': 'b1f47b31ac5fb11d45b17576d57775931191da73bec566b02074a119bb6700fd', 'stego_sha256': '10aa54fdbce9fa694
Original logo baseline matches: True
Original profile baseline matches: True
Logo original and stego identical: False
Profile original and stego identical: False
C:\Users\rkory\AppData\Local\Temp\ipykernel_9824\3261477314.py:3: DeprecationWarning: Image.Image.getdata is deprecated and will be removed in Pillow 14 (2027-10-15). Use get_flattene
  a = list(original.convert("RGBA").getdata())
C:\Users\rkory\AppData\Local\Temp\ipykernel_9824\3261477314.py:4: DeprecationWarning: Image.Image.getdata is deprecated and will be removed in Pillow 14 (2027-10-15). Use get_flattene
  b = list(stego.convert("RGBA").getdata())
```

If two images look identical but their hashes differ, does that prove one contains a hidden message? Why or why not? What additional evidence would you examine?

No. Different SHA-256 hashes only prove that the file bytes are different. They do not prove that one image contains a hidden message. The differences could be caused by ordinary image editing, metadata changes, or saving the image in a different format. I would examine the pixel level differences, compare the original and modified image properties, and use an appropriate

extraction method to check whether a hidden message can be recovered. I would also verify whether the recovered message is consistent with the suspected encoding method.

5A. SHA-256 hash comparison of the original and steganographic logo and profile images, showing whether the files changed after LSB and Stepic encoding.

```python
# Task 5A — compare each original with its stego-image.
logo_stats = compare_images(LOGO, LOGO_STEGO)
profile_stats = compare_images(PROFILE, profile_stego)

print("Original logo baseline matches current original:",
      original_logo_hash == sha256_file(LOGO))
print("Original profile baseline matches current profile:",
      original_profile_hash == sha256_file(PROFILE))
print("Logo original and stego identical:",
      original_logo_hash == sha256_file(LOGO_STEGO))
print("Profile original and stego identical:",
      original_profile_hash == sha256_file(profile_stego))

print("\nLogo comparison:", logo_stats)
print("Logo changed pixels: %.4f%%" % (100 * logo_stats["changed_pixels"] / logo_stats["total_pixels"]))
print("\nProfile comparison:", profile_stats)
print("Profile changed pixels: %.4f%%" % (100 * profile_stats["changed_pixels"] / profile_stats["total_pixels"]))
```

✓ 1.7s Python

```
C:\Users\rkory\AppData\Local\Temp\ipykernel_9824\3261477314.py:3: DeprecationWarning: Image.Image.getdata is deprecated and will be removed in Pillow 14 (2027-10-15). Use get_flattene
  a = list(original.convert("RGBA").getdata())
C:\Users\rkory\AppData\Local\Temp\ipykernel_9824\3261477314.py:4: DeprecationWarning: Image.Image.getdata is deprecated and will be removed in Pillow 14 (2027-10-15). Use get_flattene
  b = list(stego.convert("RGBA").getdata())
Original logo baseline matches current original: True
Original profile baseline matches current profile: True
Logo original and stego identical: False
Profile original and stego identical: False

Logo comparison: {'original_bytes': 1029126, 'stego_bytes': 816889, 'original_sha256': '64303ae8e227a463b9d1c9c27359d228bf0874c1284535f6f80f3e0802a6a6e0', 'stego_sha256': 'a8634a1d377
Logo changed pixels: 0.0037%

Profile comparison: {'original_bytes': 2424849, 'stego_bytes': 1902331, 'original_sha256': '292d9a5bcf8b10017abc1e633a86ae0ddbac4e5c2156e84d6dbf5307b3df5850', 'stego_sha256': '0c9913c
Profile changed pixels: 0.0040%
```

The SHA-256 hash of each original image differs from the hash of its stego-image, which shows the file bytes changed when the message was embedded. The logo changed in 39 of 1,048,576 pixels (0.0037%) and the profile changed in 43 of 1,081,344 pixels (0.0040%), so only a very small part of each image was altered, which is why the two versions look identical. Hashing each stego-image again after extraction gave the same hash as before extraction, which shows that reading the message did not change the file.

A different hash does not by itself prove that an image contains a hidden message. Any change to the file, such as re-saving it, changes its hash. The hash only proves the bytes are different. The additional evidence is the pixel comparison, which shows a small number of changed pixels, and the extraction itself, which recovered the exact messages that were embedded.

Why does it matter that extraction does not modify the image?
It matters because extraction should read the hidden information without changing the original steganographic image. If the hash before and after extraction matches, it supports the conclusion that the file's bytes remained unchanged during extraction. This helps preserve the integrity of the evidence. The recovered message should still be checked separately because an unchanged hash does not prove that the message is trustworthy.

5B. Verification of LSB and Stepic extraction integrity using SHA-256 hashes before and after extraction, confirming that neither extraction process modified its steganographic image.

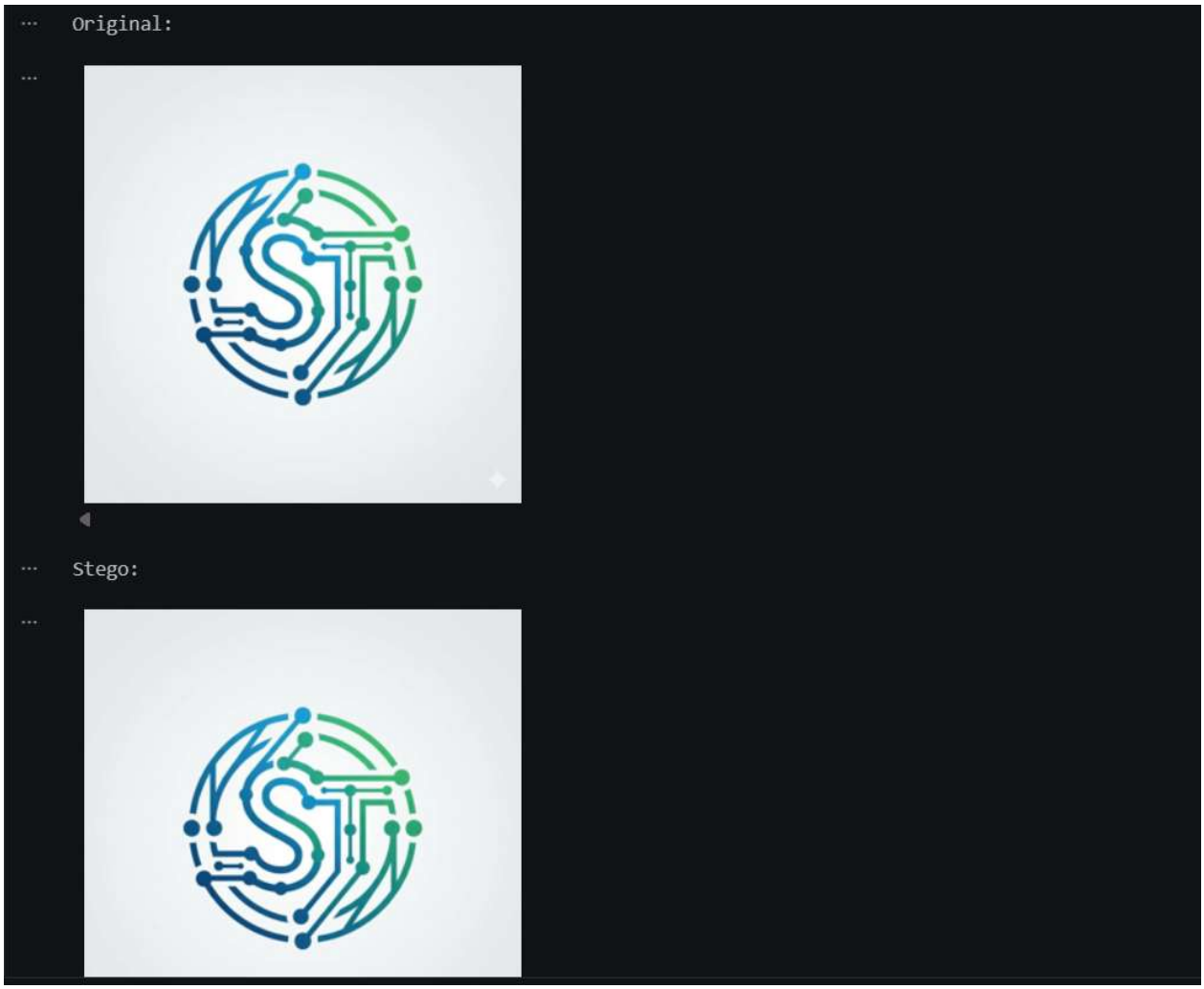
```
    print("Stepic extraction did not modify the image.")

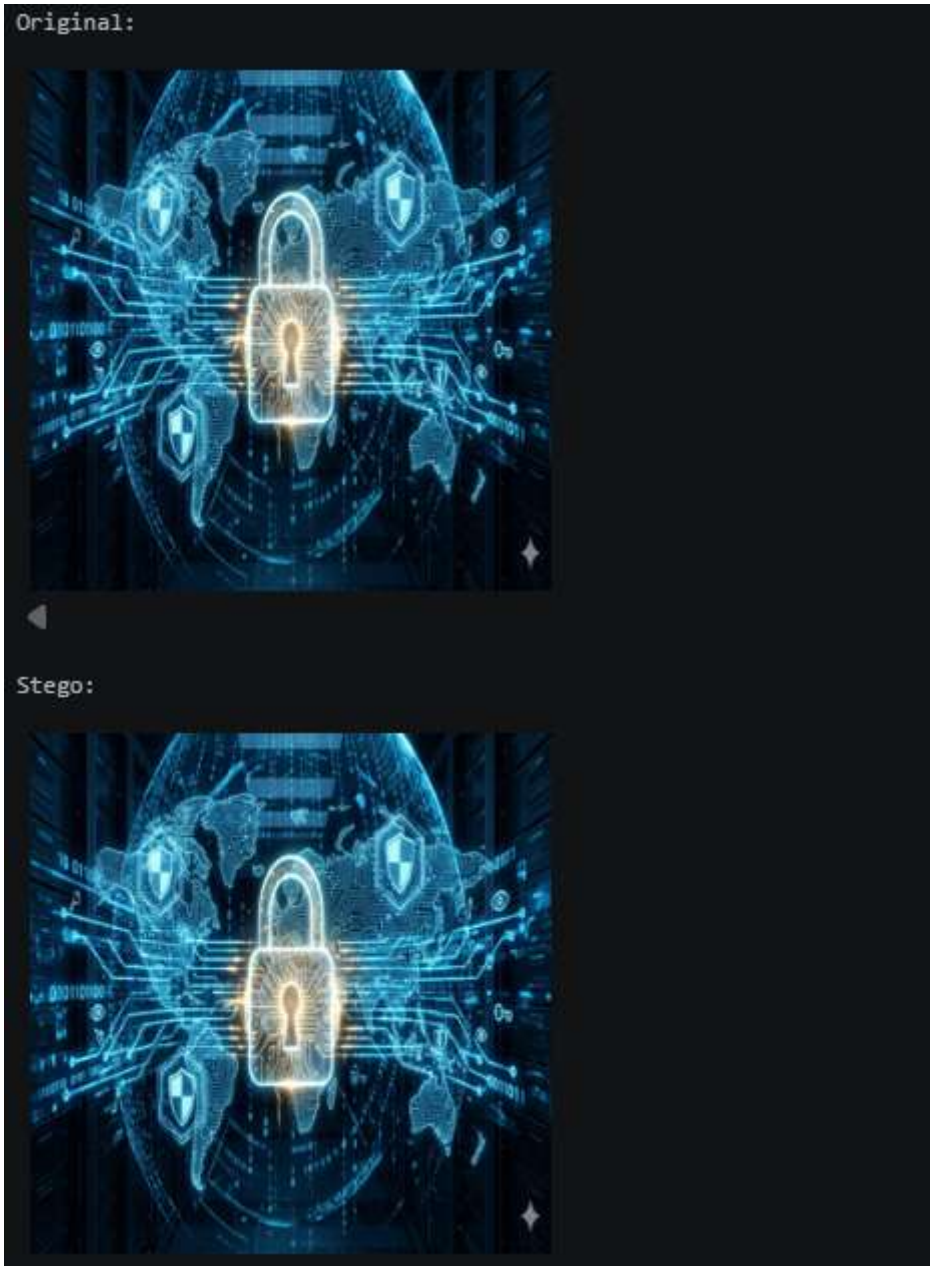
]  ✓ 0.1s

LSB recovered message: TARGET:192.168.1.50
LSB hash before extraction: a8634a1d37758640a45d211b802d569b981dcd22cfbf89b4d84a54b9738c8638
LSB hash after extraction: a8634a1d37758640a45d211b802d569b981dcd22cfbf89b4d84a54b9738c8638
LSB extraction did not modify the image.

Stepic recovered port list: 80,443,3478,5060
Stepic hash before extraction: 0c9913c5322e00d97d6bce575520598c805882bee17aa3819967f697f927dbad
Stepic hash after extraction: 0c9913c5322e00d97d6bce575520598c805882bee17aa3819967f697f927dbad
Stepic extraction did not modify the image.
```

5C. Visual comparison of original and steganographic logo and profile images. The comparison demonstrates how LSB embedding can preserve the images' overall visual appearance.





Part 6

Least Significant Bit steganography hides a message inside an image. Encryption scrambles a message so it needs a key to read. Anyone who knows which bits to read and where the message ends can read the hidden payload without a key. I recovered both payloads exactly. My manual decoder and the supplied decoder both returned TARGET:192.168.1.50 from the logo. Stepic returned the port list 80,443,3478,5060 from the profile image.

Capacity depends on image size, the number of colour channels, and the space needed for the terminator. A 100 by 100 RGBA image holds 4,999 bytes. The same image in RGB holds 3,749 bytes, with three channels instead of four. My payload was 19 characters, so with the terminator it needed 20 bytes, or 40 pixels. PNG works here because it is lossless, so exact pixel values survive saving. JPEG is lossy, so saving changes pixel values and can destroy the least significant bits and the message with them.

The SHA-256 hashes changed between each original and its stego-image, because changing the least significant bits changes the file bytes. The logo changed in 39 of 1,048,576 pixels (0.0037%) and the profile in 43 of 1,081,344 (0.0040%), which is why the images still look identical in Part 5C. Hashing each stego-image after extraction gave the same hash, so reading the message did not change either file. A different hash does not prove a hidden message, because any change to a file changes its hash. The pixel comparison and the extraction are the extra evidence.

This method has two limits: it hides only a small amount of data, and it breaks if the image is saved with lossy compression. In a forensic investigation I would compare the suspect image with the original, if one is available, and try to extract a message with the suspected method. Finally, the TARGET label and the port list are not evidence of a real scan or theft. They are strings I embedded myself. Showing they were ever used would need host logs, proxy records, or packet captures